# Programación en GPUs con CUDA
# Laboratorio 3 — Producto de matrices (GEMM)

## 1. Objetivos del laboratorio

En este laboratorio implementaremos progresivamente el producto de matrices en GPU:

$$\mathbf{C} = \mathbf{AB}$$

El objetivo no es solo obtener una implementación correcta, sino comprender:

- Cómo mapear trabajo a hilos CUDA.
- Cómo influye la organización de la grid y los bloques.
- Por qué la reutilización de datos mejora el rendimiento.
- Cómo la memoria compartida permite acelerar GEMM.

## 2. Producto de matrices

Sean:

- $\mathbf{A}$ de tamaño $h_A \times w_A$,
- $\mathbf{B}$ de tamaño $w_A \times w_B$,
- $\mathbf{C}$ de tamaño $h_A \times w_B$.

Cada elemento de $\mathbf{C}$ se calcula como:

$$C_{ij} = \sum_{k=0}^{w_A-1} A_{ik} B_{kj}$$

En memoria lineal (row-major):

```
C[row*wB + col] += A[row*wA + k] * B[k*wB + col];
```

El número aproximado de operaciones es:

$$2 \cdot h_A \cdot w_A \cdot w_B$$

Definiremos:

$$GFLOP/s = \frac{2 h_A w_A w_B}{t \cdot 10^9}$$

donde $t$ es el tiempo de ejecución.

## 3. Intensidad aritmética y por qué GEMM es especial

A diferencia de la transposición de matrices (que es claramente *memory-bound*), el producto de matrices presenta una característica muy importante: una alta **intensidad aritmética**.

### ¿Qué es la intensidad aritmética?

Se define como:

$$\text{Intensidad aritmética} = \frac{\text{Operaciones en coma flotante}}{\text{Bytes transferidos desde memoria}}$$

En el producto de matrices:

- Cada elemento cargado de A puede reutilizarse varias veces.
- Cada elemento cargado de B puede reutilizarse varias veces.

Si utilizamos tiling correctamente, un mismo dato cargado desde memoria global puede emplearse en múltiples multiplicaciones.

### Consecuencia

Mientras que la transposición es principalmente limitada por ancho de banda, la multiplicación de matrices puede llegar a estar limitada por:

- capacidad de cómputo (FLOP/s),
- o eficiencia en el uso de memoria compartida.

Por este motivo, GEMM (General Matrix Multiply) es una operación fundamental en:

- HPC
- Simulación científica
- Machine Learning
- Deep Learning

De hecho, la mayoría de bibliotecas numéricas optimizadas dedican un gran esfuerzo a optimizar esta operación.

## 4. Figuras de referencia (Guía CUDA)

Las Figuras 8 y 9 de la guía oficial de CUDA ilustran:

- Multiplicación sin shared memory.
- Multiplicación con tiling en shared memory.

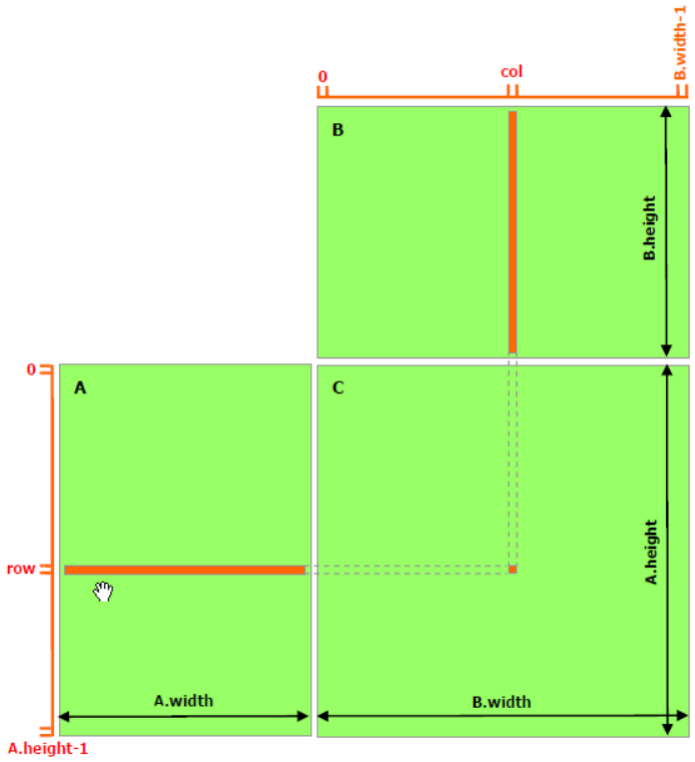


Figura 1: Esquema sin memoria compartida.

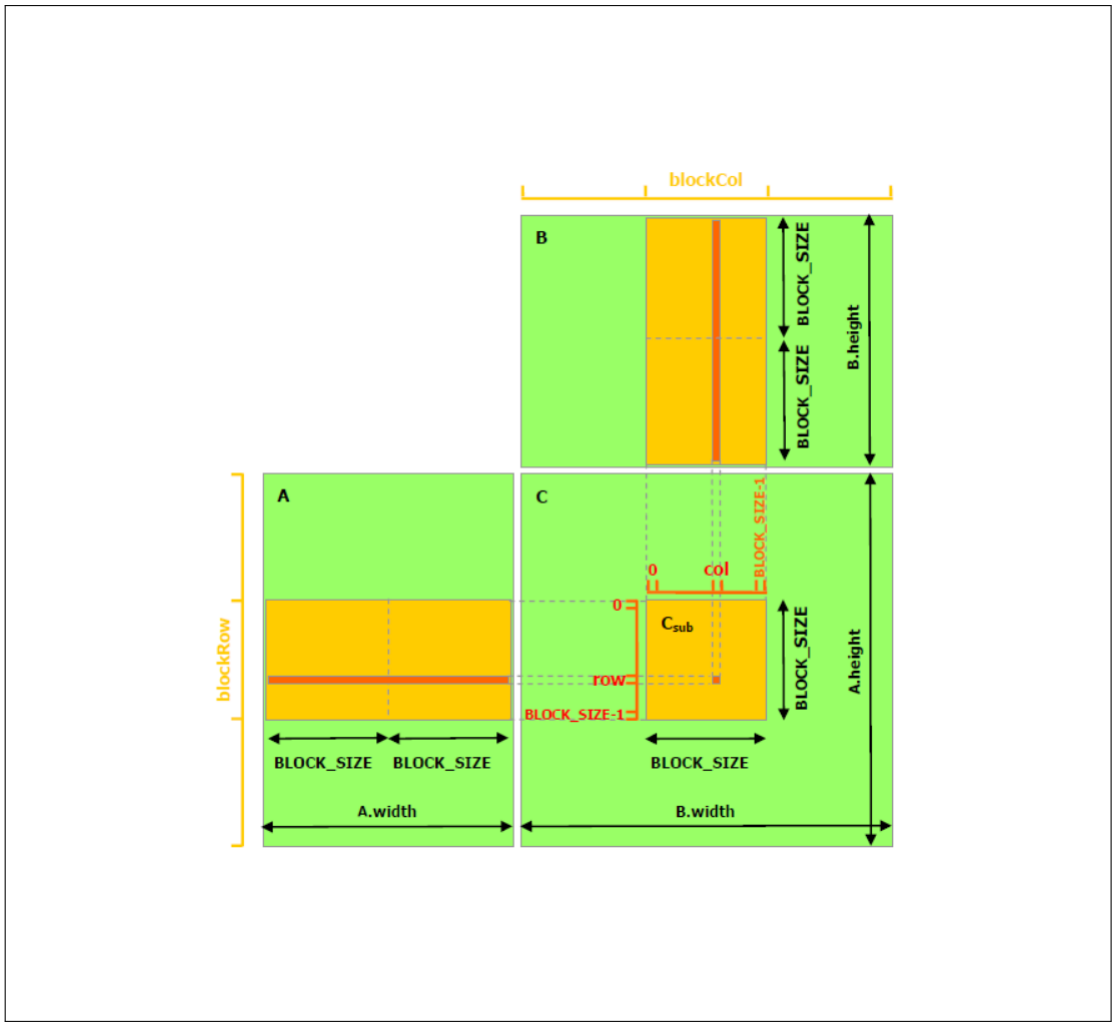


Figura 2: Esquema con tiling y memoria compartida.

# 5. Ejercicio 1 — v0: Un solo hilo calcula toda la matriz

## 5.1. Objetivo

Conseguir una implementación CUDA correcta usando un único hilo.

## 5.2. Instrucciones

- Configurar `dimGrid(1,1)` y `dimBlock(1,1)`.
- Implementar en el kernel los tres bucles i, j, k.

## 5.3. Kernel (a completar)

```
__global__ void Muld(float* A, float* B,
                     int hA, int wA, int wB,
                     float* C)
{
  for (int row = 0; row < hA; row++) {
    for (int col = 0; col < wB; col++) {
      float acc = 0.0f;
      for (int k = 0; k < wA; k++) {
```

```
        // COMPLETAR
      }
      // COMPLETAR escritura en C
    }
  }
}
```

**Comprobación:** verificar que no aparece error en `diff()`.

# 6. Ejercicio 2 — v1: Un hilo por elemento

## 6.1. Objetivo

Paralelizar para que cada hilo calcule un elemento $C_{row,col}$.

## 6.2. Instrucciones

- Usar grid 2D.
- Calcular row y col con blockIdx y threadIdx.
- Añadir control de límites.
- Probar distintos tamaños de bloque (8x8, 16x16, 32x8).

## 6.3. Kernel (a completar)

```
__global__ void Muld(float* A, float* B,
                     int hA, int wA, int wB,
                     float* C)
{
  int col = blockIdx.x * blockDim.x + threadIdx.x;
  int row = blockIdx.y * blockDim.y + threadIdx.y;

  if (row < hA && col < wB) {
    float acc = 0.0f;
    for (int k = 0; k < wA; k++) {
      // COMPLETAR
    }
    C[row*wB + col] = acc;
  }
}
```

## 6.4. Entrega

Tabla con tiempo y GFLOP/s para varios tamaños.

# 7. Ejercicio 3 — v2: Tiling con shared memory

## 7.1. Objetivo

Implementar versión con memoria compartida.

### 7.2. Kernel base

```
#define BLOCK_SIZE 16

__global__ void Muld(float* A, float* B,
                     int hA, int wA, int wB,
                     float* C)
{
  int tx = threadIdx.x;
  int ty = threadIdx.y;

  int col = blockIdx.x * BLOCK_SIZE + tx;
  int row = blockIdx.y * BLOCK_SIZE + ty;

  __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
  __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];

  float acc = 0.0f;

  for (int t = 0; t < numTiles; t++) {
        // Completar


        // Completar

  }

  if (row < hA && col < wB)
    C[row*wB + col] = acc;
}
```

### 7.3. Entrega

Comparar v1 y v2 para distintos tamaños.

## 8. Ejercicio opcional — Soporte a tamaños no múltiplos

Modificar la grid:

```
dim3 dimGrid((wB + BLOCK_SIZE - 1) / BLOCK_SIZE,
             (hA + BLOCK_SIZE - 1) / BLOCK_SIZE);
```

Asegurarse de:

- Comprobar límites en cargas a shared.
- Comprobar límites en escritura final.

### 8.1. Pregunta

¿Por qué es necesario cargar 0.0f cuando el índice está fuera de rango?

## 9. Ejercicio opcional avanzado — Comparación con cuBLAS

CUDA proporciona la biblioteca altamente optimizada **cuBLAS**, que implementa operaciones BLAS en GPU.

En particular, el producto de matrices en precisión simple se implementa mediante:

```
cublasSgemm(...)
```

### Objetivo

Comparar el rendimiento de vuestra mejor versión (v2) con la implementación oficial de cuBLAS.

### Pasos

1. Incluir:

   ```
   #include <cublas_v2.h>
   ```

2. Crear un handle:

   ```
   cublasHandle_t handle;
   cublasCreate(&handle);
   ```

3. Llamar a:

   ```
   cublasSgemm(handle,
               CUBLAS_OP_N,
               CUBLAS_OP_N,
               wB, hA, wA,
               &alpha,
               Bd, wB,
               Ad, wA,
               &beta,
               Cd, wB);
   ```

### Nota importante

cuBLAS utiliza almacenamiento **column-major**. Dado que nuestras matrices están en **row-major**, la forma más sencilla de adaptarlo es intercambiar A y B en la llamada.

### Medición

Medir el tiempo del kernel cuBLAS y calcular GFLOP/s.

### Preguntas

1. ¿Cuánto más rápido es cuBLAS que vuestra versión?
2. ¿Por qué una librería especializada puede ser significativamente más rápida?
3. ¿Qué optimizaciones creéis que puede estar utilizando internamente?

## 10. Análisis

Generar:

- GFLOP/s vs tamaño.
- Tiempo kernel vs tamaño.

| Tamaño | Versión | Tiempo (ms) | GFLOP/s | Bloque |
|---|---|---|---|---|
| | v0 | | | |
| | v1 | | | |
| | v2 | | | |

## 11. Conclusiones

Discutir:

- Diferencias entre v0, v1 y v2.
- Impacto de la paralelización.
- Impacto de la reutilización en shared.